
CS2306 Lab4

简单的类 MIPS 单周期处理器存储部件的设计与实现（二）

姓名： 吴恒涛 学号： 524031910038

摘要

本报告详述了简单的类 MIPS 单周期处理器中核心存储与数据扩展部件的设计实现与仿真验证。实验在 Vivado 环境下, 以 xc7k325tffg676-2 为目标 FPGA 平台, 利用 Verilog HDL 对寄存器组 (Registers)、数据存储器 (Data Memory) 以及带符号扩展单元 (Sign Extend) 进行了行为级建模。重点分析了利用时钟下降沿防止读写冲突的同步策略, 并通过全面的 Testbench 覆盖验证了各模块的逻辑正确性, 为后续构建完整的数据通路奠定了基础。

目录

1 实验目的与引言	2
2 核心部件设计原理	2
2.1 寄存器堆 (Register File) 读写时序原理	2
2.2 数据存储器 (Data Memory) 访问机制	2
2.3 带符号扩展 (Sign Extension) 原理	3
3 硬件架构与核心逻辑实现	3
3.1 寄存器组模块 (Registers)	3
3.2 数据存储器模块 (Data Memory)	3
3.3 带符号扩展模块 (Sign Extend)	4
4 测试平台构造与行为级仿真自查	4
4.1 寄存器组模块 (Registers) 波形自查	4
4.2 数据存储器模块 (Data Memory) 波形自查	5
4.3 带符号扩展模块 (Sign Extend) 波形自查	5
5 工程反思与总结	6

1 实验目的与导言

本次实验的预计完成时间为 150 分钟，旨在深入理解处理器内部数据存储与格式转换机制。具体实验目的如下：

- 理解寄存器、数据存储器、有符号扩展单元的 I/O 定义。
- Registers 的设计实现。
- Data Memory 的设计实现。
- 有符号扩展部件的实现。
- 对功能模块进行仿真。

2 核心部件设计原理

单周期 MIPS 处理器的本质是各个功能部件在全局统一时钟节拍下的协同工作。本次实验聚焦于数据通路中的三大基础数据处理与存储部件，其底层设计原理如下：

2.1 寄存器堆 (Register File) 读写时序原理

在 MIPS 体系结构中，寄存器堆由 32 个 32 位宽的通用寄存器构成，其中 0 号寄存器 (\$zero) 的值被硬件级恒定约束为 0。在单周期架构下，寄存器堆的读写操作存在严格的时序隔离要求：

- **异步读机制：**为了保证组合逻辑数据流的畅通，寄存器的读操作不依赖时钟边沿。只要读地址发生有效跳变，寄存器堆应当立即将对应地址的数据输出到读数据总线上。
- **同步写机制：**写操作具备状态修改的破坏性，必须与时钟信号同步。通常采用时钟下降沿 (negedge) 触发写操作，确保在时钟周期的前半段完成指令译码和数据读取，在后半段的安全窗口内将运算结果稳定写入目标寄存器，避免“写后读 (RAW)”冒险。

2.2 数据存储器 (Data Memory) 访问机制

与寄存器堆类似，数据存储器的读取操作通常设计为响应地址信号的纯组合逻辑，而写入操作必须受到写使能信号 (memWrite) 和全局时钟的严格选通控制。本实验中为了简化地址映射，将物理存储单元抽象为深度为 64 的 32 位字对齐存储阵列。

2.3 带符号扩展 (Sign Extension) 原理

在处理包含 16 位立即数 (Immediate) 的 I 型指令时, ALU 运算部件要求操作数必须达到 32 位位宽。根据补码表示法则, 对有符号数进行位宽扩展时, 必须保持其数值和符号的恒定: 即硬件层面的带符号扩展逻辑实质上是将 16 位输入数据的最高位进行复制, 并拼接至目标位宽的高位区间。

3 硬件架构与核心逻辑实现

3.1 寄存器组模块 (Registers)

寄存器是指令操作的主要对象。在读写逻辑的设计中, 由于不确定 WriteReg、WriteData、RegWrite 信号的先后次序, 采用时钟的下降沿作为写操作的同步信号。

```
// 声明 32 个 32 位的寄存器文件
reg [31:0] regFile [31:0];

// 异步读操作
always @(readReg1 or readReg2 or writeReg) begin
    readData1 = (readReg1 == 5'b0) ? 32'b0 : regFile[readReg1];
    readData2 = (readReg2 == 5'b0) ? 32'b0 : regFile[readReg2];
end

// 同步写操作: 在时钟下降沿触发
always @(negedge Clk) begin
    if (regWrite && (writeReg != 5'b0)) begin
        regFile[writeReg] <= writeData;
    end
end
```

3.2 数据存储器模块 (Data Memory)

Data memory 是用来存储运行完成的数据。由于写数据也要考虑信号同步, 因此也需要接入时钟信号。此处构建了一个深度为 64 的 32 位存储阵列。

```
// 声明深度为 64 的 32 位内存文件
reg [31:0] memFile [0:63];
```

```

// 内存读操作
always @(*) begin
    if (memRead)
        readData = memFile[address[5:0]];
    else
        readData = 32'b0;
end

// 内存写操作：下降沿同步
always @(negedge Clk) begin
    if (memWrite) begin
        memFile[address[5:0]] <= writeData;
    end
end
end

```

3.3 带符号扩展模块 (Sign Extend)

该模块的功能是将 16 位有符号数扩展为 32 位有符号数。利用 Verilog 的位拼接与复制语法可实现硬件描述：

```

module signext(
    input  [15:0] inst,
    output [31:0] data
);

// 提取 16 位输入的最高位（符号位），复制 16 次拼接在高位
assign data = {{16{inst[15]}}, inst};

endmodule

```

4 测试平台构造与行为级仿真自查

4.1 寄存器组模块 (Registers) 波形自查

在 Registers_tb 测试文件中, 设置时钟周期为 200ns。波形显示, 当且仅当 regWrite 为高电平且遇到 Clk 下降沿时, 数据才被安全写入对应的寄存器槽位。

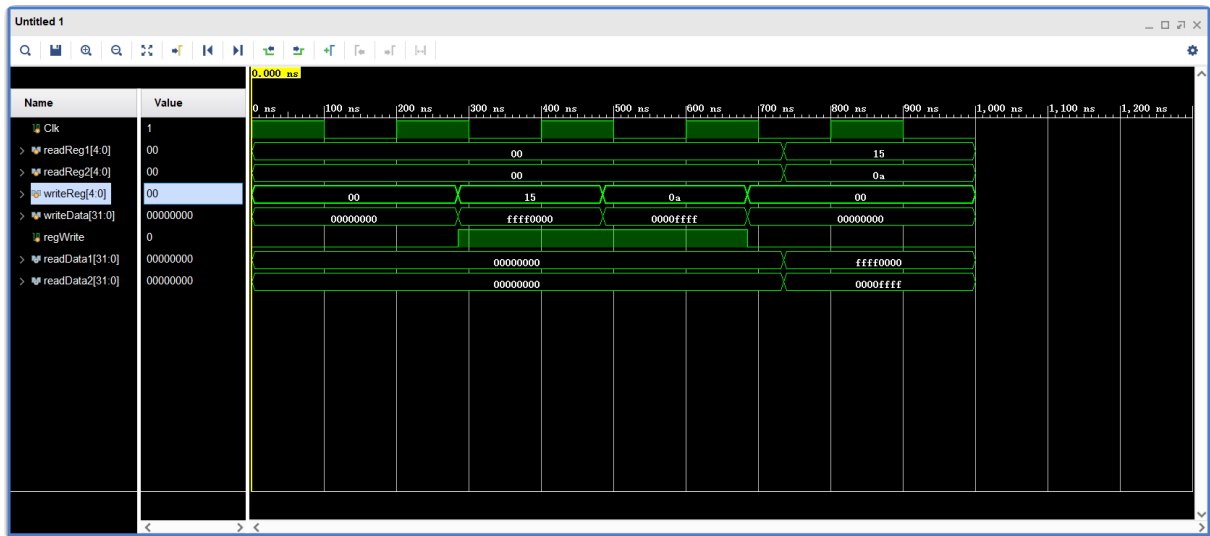


图 1: Registers 模块仿真波形图

4.2 数据存储器模块 (Data Memory) 波形自查

针对数据存储器，在 dataMemory_tb 中添加了激励信号。通过波形可以观察到，在写使能（memWrite）有效时，数据被成功同步写入到指定地址单元。

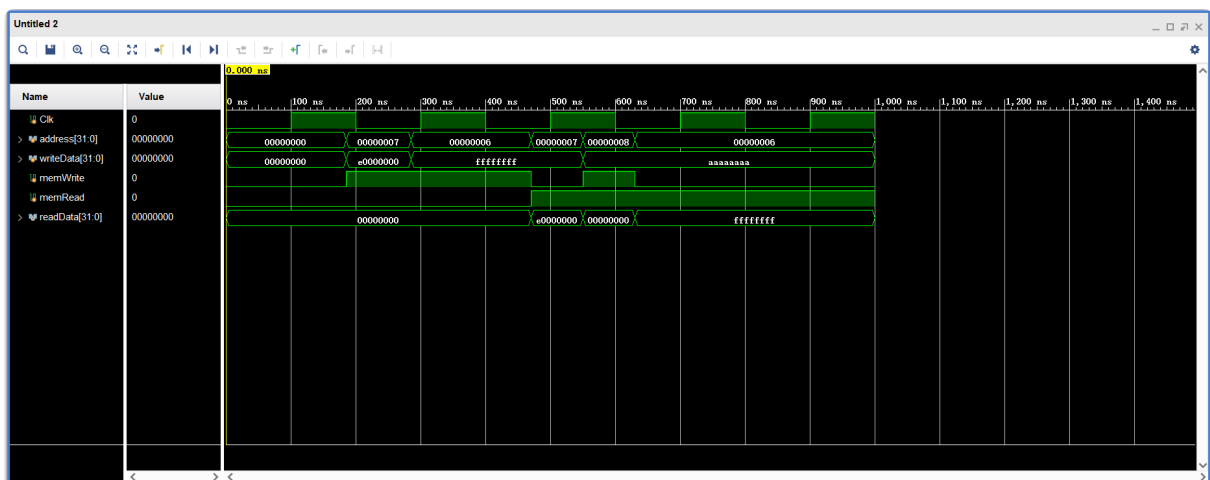


图 2: Data Memory 模块读写时序仿真波形图

4.3 带符号扩展模块 (Sign Extend) 波形自查

从波形上可见，当输入最高位为 1 时，扩展后的 32 位数据高 16 位全为 1；最高位为 0 时则全部补 0，完全符合符号位扩展准则。

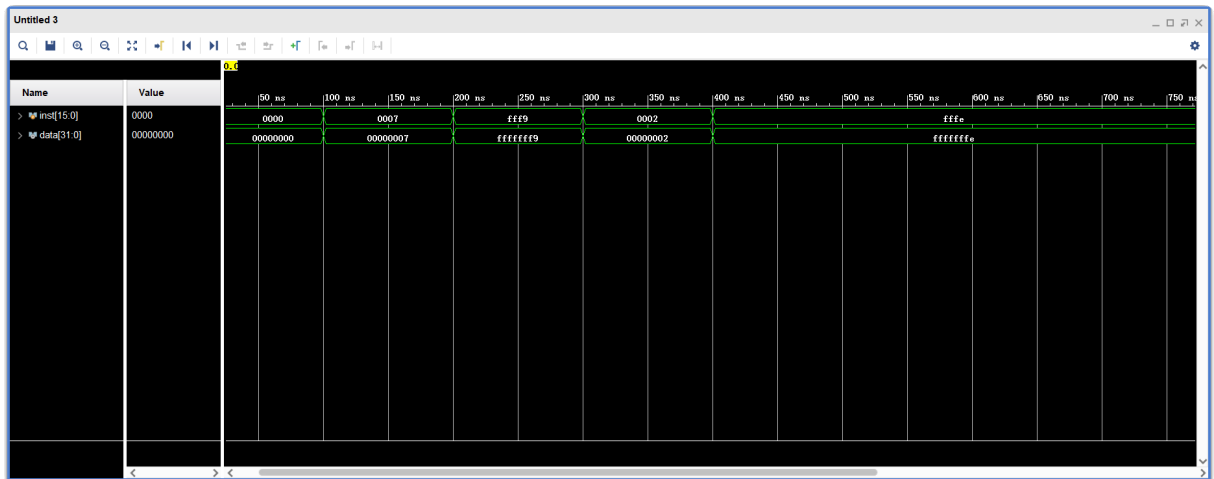


图 3: Sign Extend 模块正负数符号扩展行为仿真波形图

5 工程反思与总结

本次实验通过严谨的硬件描述语言设计，成功实现了处理器中至关重要的数据缓存与格式化部件。在 Registers 和 Data Memory 的开发中，我深刻体会到了时序边界设计的重要性。利用 `negedge Clk` 进行写同步，巧妙地避免了同一时钟周期内潜在的读写冲突冒险。此外，有符号扩展逻辑的实现也体现了 Verilog 位拼接操作在硬件映射上的高效性。这三个基础模块的正确仿真，为单周期系统整机数据通路联调提供了可靠的底层保障。